

Recursividade

COMP0496 – Programação A

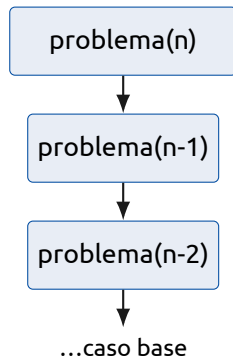
Prof. Dr. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
`andreyoshiaki@dcomp.ufs.br`

Uma matrioska contém uma versão menor de si mesma,
que contém uma versão menor ainda,
até chegar à menor boneca indivisível.

Problemas recursivos têm a mesma estrutura:

- Reduzem-se a **versões menores** do mesmo problema
- Têm uma **condição de parada** (a menor boneca)



Recursividade

Uma função é **recursiva** quando chama a si mesma. Toda função recursiva precisa de dois elementos:

1. **Caso base:** condição de parada — sem ele, recursão infinita.
2. **Chamada recursiva:** problema reduzido a uma versão menor.

```
1 def funcao_recursiva(n):  
2     if <caso base>:           # condicao de parada  
3         return <resultado base>  
4     return <combinar> funcao_recursiva(<n reduzido>)
```

Característica: uma única chamada recursiva que reduz o problema por um passo.

```
1  def soma_digitos(n):
2      if n < 10:                # caso base: um so digito
3          return n
4      return n % 10 + soma_digitos(n // 10)
5
6  # Tracando soma_digitos(123):
7  # soma_digitos(123) = 3 + soma_digitos(12)
8  # soma_digitos(12)  = 2 + soma_digitos(1)
9  # soma_digitos(1)   = 1                <- caso base
10 # Resultado: 3 + 2 + 1 = 6
11 print(soma_digitos(123)) # 6
12 print(soma_digitos(9999)) # 36
```

Característica: estrutura arbitrária — pode gerar múltiplas chamadas recursivas.

```
1  def flatten(lista):
2      """Achata uma lista com sublistas aninhadas."""
3      resultado = []
4      for item in lista:
5          if isinstance(item, list):    # item e uma lista?
6              resultado += flatten(item) # chamar recursivamente
7          else:
8              resultado.append(item)    # item simples: adicionar
9      return resultado
10
11 dados = [1, [2, [3, 4]], 5, [6]]
12 print(flatten(dados))    # [1, 2, 3, 4, 5, 6]
13
14 aninhado = [[1, 2], [3, [4, [5]]]]
15 print(flatten(aninhado)) # [1, 2, 3, 4, 5]
```

Problema da recursão primitiva: cada chamada empilha um frame na memória.

Solução: usar um **acumulador** — o resultado é construído no parâmetro, não na volta.

```
1  # Versao com acumulador (recursao na cauda)
2  def inverter(s, acc=""):
3      if s == "":                # caso base: string vazia
4          return acc
5      return inverter(s[1:], s[0] + acc)
6
7  # Tracando inverter("abc"):
8  # inverter("abc", "") -> inverter("bc", "a")
9  # inverter("bc", "a") -> inverter("c", "ba")
10 # inverter("c", "ba") -> inverter("", "cba")
11 # inverter("", "cba") -> "cba" <- caso base
12 print(inverter("python")) # "nohtyp"
```

Característica: duas ou mais funções que se chamam mutuamente.

```
1  def is_par(n):
2      if n == 0:
3          return True
4      return is_impar(n - 1)
5
6  def is_impar(n):
7      if n == 0:
8          return False
9      return is_par(n - 1)
10
11  # Tracando is_par(3):
12  # is_par(3) -> is_impar(2) -> is_par(1)
13  #           -> is_impar(0) -> False
14  print(is_par(4))    # True
15  print(is_impar(3)) # True
```

Padrão	Característica	Exemplo
Primitiva	Uma chamada, redução direta	soma_digitos
Geral	Estrutura arbitrária, múltiplas chamadas	flatten
Cauda	Acumulador evita empilhamento	inverter
Mútua	Duas funções interdependentes	is_par/is_impar

Em todos os padrões: o **caso base** é obrigatório. Sem ele, a recursão não termina.

- Toda função recursiva precisa de **caso base** (parada) e **chamada recursiva** (redução).
- **Primitiva**: redução direta passo a passo — ex: soma de dígitos.
- **Cauda**: acumulador constrói resultado no parâmetro — evita empilhamento.
- **Mútua**: duas funções que se chamam — cada uma define metade do caso base.